

Backend Setup (Node.js & Express.js)

The backend of the **DarshanEase – Temple Darshan Ticket Booking System** is developed using **Node.js** and **Express.js**. It is responsible for handling business logic, database operations, authentication, and REST API services. The following steps are used to set up the backend application.

Step 1: Open the Server Folder

Open the **DarshanEase** project in **Visual Studio Code (VS Code)**. Open the integrated terminal and navigate to the **server** folder:

```
cd server
```

Step 2: Initialize the Node.js Project

Initialize the backend project by creating a **package.json** file using the following command:

```
npm init -y
```

This command generates a default **package.json** file containing the project's basic configuration.

Step 3: Create the Main Server File

Create the main backend entry file:

```
server.js
```

This file initializes the Express server, connects to the MongoDB database, configures middleware, and starts the backend application.

Step 4: Create the Required Project Folders

Organize the backend source code by creating the following folders inside the **server** directory:

```
server/
|
|— config/
|— controllers/
|— middleware/
|— models/
|— routes/
|— services/
|— utils/
|— uploads/
|— node_modules/
```

Folder Description

- **config/** – Contains database connection, JWT configuration, and Cloudinary configuration files.
 - **controllers/** – Contains the business logic for user authentication, temple management, bookings, payments, and admin operations.
 - **middleware/** – Stores middleware functions for authentication, authorization, file uploads, and error handling.
 - **models/** – Contains Mongoose schema definitions for User, Temple, Booking, Darshan Slot, and Payment collections.
 - **routes/** – Defines REST API endpoints and connects them to controller functions.
 - **services/** – Contains reusable services such as booking management, payment processing, and email notifications.
 - **utils/** – Stores utility functions like token generation, response formatting, and input validation.
 - **uploads/** – Stores uploaded temple images, payment receipts, and other files.
-

Step 5: Project Structure

```
server/
|
|— config/
```

- |— controllers/
- |— middleware/
- |— models/
- |— routes/
- |— services/
- |— utils/
- |— uploads/
- |— server.js
- |— package.json

Backend Technologies Used

- **Node.js** – JavaScript Runtime Environment
- **Express.js** – Backend Framework
- **MongoDB** – NoSQL Database
- **Mongoose** – MongoDB Object Modeling
- **JWT (JSON Web Token)** – User Authentication
- **bcrypt.js** – Password Encryption
- **Multer** – File Upload Handling
- **Cloudinary** – Cloud Image Storage
- **dotenv** – Environment Variable Management